

Chapter 1

Smart Output Documentation

1.1 Introduction

The `smart_output.py` module provides output abstraction for CTCC: it chooses `CSVOutputManager` or `JSONOutputManager` at instantiation time based on the `CTCC_OUTPUT_MODE` environment variable, then delegates all method calls to the underlying manager. Calculation scripts import `CTCCOutputManager` from `smart_output` (which is an alias for `SmartOutputManager`) and use the same `add_*`, `write_batch_summary` API regardless of mode; the backend writes either CSV (`batch_summary.csv` and related) or JSON (intermediate `json_output-<id>-<module>.json` files for aggregation by `ctcc.py`).

1.2 Method Overview

1.2.1 Purpose

1. On `CTCCOutputManager()` (i.e. `SmartOutputManager()`): read `CTCC_OUTPUT_MODE` (default `csv`); if `json`, create `JSONOutputManager(scenario_id=CTCC_SCENARIO_ID` or `‘‘default’’`); else create `csv_output_manager.CTCCOutputManager()`.
2. Expose the same API as the underlying manager: `add_build_costs`, `add_row_costs`, `write_batch_summary`, etc., via `__getattr__` delegation to `self._manager`.
3. Override `write_batch_summary`: in JSON mode, infer the calling script name (from the call stack or `sys.argv[0]` when `__name__ == ‘__main__’`), then call `self._manager.save_to_file(mo` in CSV mode, call `self._manager.write_batch_summary()`.

1.2.2 Calculation Approach

Dispatch only; no cost/benefit formulas. The module does not implement output logic; it only selects the backend and delegates. All `add_*` and write logic live in `csv_output_manager.py` or `json_output_manager.py` (see their documentation).

1.2.3 Inputs and Outputs

Inputs:

- CTCC_OUTPUT_MODE environment variable (read at `SmartOutputManager.__init__`): `csv` (default) or `json`.
- CTCC_SCENARIO_ID (optional, used in JSON mode when constructing `JSONOutputManager`).

Outputs:

- Same `CTCCOutputManager` API; backend writes CSV (`batch_summary.csv`, etc.) or JSON (intermediate files; final `ctcc_results_<id>.json` is written by `ctcc.py` after aggregation).

1.2.4 Integration with Other Scripts

- Every calculation script that writes results imports `CTCCOutputManager` from `smart_output` and calls `add_build_costs(results)`, `write_batch_summary()`, etc.
- `smart_output` delegates to `csv_output_manager.CTCCOutputManager` or `json_output_manager.JSONOutputManager`. It does not implement `add_*` or file format logic.

1.3 Key Functions

1.3.1 SmartOutputManager (exported as CTCCOutputManager)

Wrapper class that holds either a `CSVOutputManager` or `JSONOutputManager` and delegates all attribute access to it.

1.3.2 __init__()

Reads `CTCC_OUTPUT_MODE` (default `csv`). If `json`, instantiates `JSONOutputManager(scenario_id=os.environ.get('CTCC_SCENARIO_ID', 'default'))`; else instantiates `csv_output_manager.CTCCOutputManager()`. Stores the instance in `self._manager`.

Key logic:

```
1 self.output_mode = os.environ.get('CTCC_OUTPUT_MODE', 'csv').lower()
2 if self.output_mode == 'json':
3     from json_output_manager import JSONOutputManager
4     scenario_id = os.environ.get('CTCC_SCENARIO_ID', 'default')
5     self._manager = JSONOutputManager(scenario_id=scenario_id)
6 else:
7     from csv_output_manager import CTCCOutputManager
8     self._manager = CTCCOutputManager()
```

1.3.3 write_batch_summary()

In JSON mode: infers the calling script name (inspect stack; if `__name__ == '__main__'` use `os.path.basename(sys.argv[0])` without extension), then calls `self._manager.save_to_file(module_name)` and returns the result. In CSV mode: calls `self._manager.write_batch_summary()` and returns the result.

1.3.4 __getattr__(name)

Delegates to `getattr(self._manager, name)` so all `add_build_costs`, `add_row_costs`, `add_bcr_metrics`, etc., are forwarded to the underlying CSV or JSON manager.

1.4 Example

Script uses the same code for both modes:

```
1 from smart_output import CTCCOutputManager
2 output_manager = CTCCOutputManager()
3 output_manager.add_build_costs(results)
4 output_manager.write_batch_summary()
```

To force JSON mode (e.g. before instantiating):

```
1 import os
2 os.environ["CTCC_OUTPUT_MODE"] = "json"
3 os.environ["CTCC_SCENARIO_ID"] = "test"
4 from smart_output import CTCCOutputManager
5 output_manager = CTCCOutputManager()
6 output_manager.add_build_costs(results)
7 output_manager.write_batch_summary() # writes
   json_output_test_build_costs.json
```